Chương 27

Máy vector hỗ trợ lề mềm

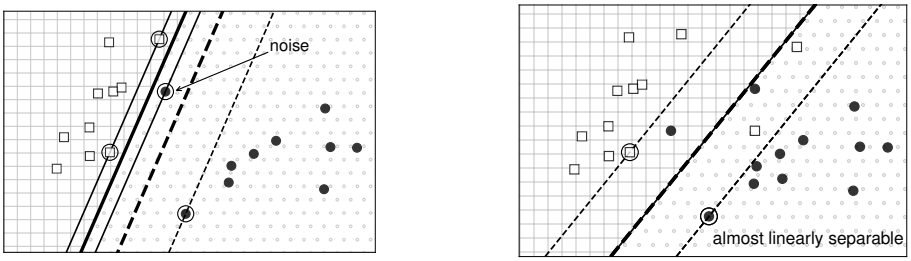
27.1. Giới thiệu

Giống với thuật toán học perceptron (PLA), máy vector hỗ trợ (SVM) chỉ làm việc khi dữ liệu của hai lớp tách biệt tuyến tính. Một cách tự nhiên, chúng ta cũng mong muốn SVM có thể làm việc với dữ liệu gần tách btệt tuyến tính như hồi quy logistic đã làm được.

Xét hai ví dụ trong Hình 27.1. Có hai trường hợp dễ nhận thấy SVM làm việc không hiệu quả hoặc thậm chí không làm việc:

- Trường hợp 1: Dữ liệu vẫn tách biệt tuyến tính như Hình 27.1a nhưng có một điểm nhiễu của lớp tròn ở quá gần lớp vuông. Trong trường hợp này, SVM sẽ tạo ra lề rất nhỏ. Ngoài ra, mặt phân cách nằm quá gần các điểm vuông và xa các điểm tròn. Trong khi đó, nếu *hy sinh* điểm nhiễu này thì ta thu được nghiệm là đường nét đứt đậm. Nghiệm này tạo ra lề rộng hơn, có khả năng tăng độ chính xác cho mô hình.

- Trường hợp 2: Dữ liệu gần tách biệt tuyến tính như trong Hình 27.1b. Trong trường hợp này, không tồn tại đường thẳng nào hoàn toàn phân chia hai lớp dữ liệu, vì vậy bài toán tối ưu SVM vô nghiệm. Tuy nhiên, nếu chấp nhận việc những điểm ở gần khu vực ranh giới bị phân loại lỗi, ta vẫn có thể tạo được một đường phân chia khá tốt như đường nét đứt đậm. Các đường hỗ trợ (nét đứt mảnh) vẫn giúp tạo được lề lớn. Với mỗi điểm nằm *lần* sang phía bên kia của các đường hỗ trợ tương ứng, ta gọi điểm đó rơi vào *vùng không an toàn*. Như trong hình, hai điểm tròn nằm phía bên trái đường hỗ trợ của lớp tròn được xếp vào loại không an toàn, mặc dù có một điểm tròn vẫn nằm trong khu vực nền chấm. Hai điểm vuông ở phía phải của đường hỗ trợ của lớp tương ứng thậm chí đều lấn sang phần có nền chấm.



(a) Khi có nhiễu nhỏ.

(b) Khi dữ liệu *gần linearly separable*.

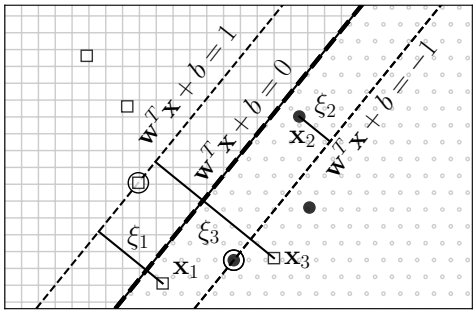
Hình 27.1. Hai trường hợp khi SVM *thuần* làm việc không hiệu quả. (a) Hai lớp vẫn tách biệt tuyến tính nhưng một điểm thuộc lớp này quá gần lớp kia, điểm này có thể là nhiễu. (b) Dữ liệu hai lớp gần tách biệt tuyến tính.

Trong cả hai trường hợp trên, lề tạo bởi đường phân chia và đường nét đứt mảnh được gọi là *lề mềm* (soft-margin). Từ *mềm* thể hiện sự linh hoạt, có thể chấp nhận việc một vài điểm bị phân loại sai để mô hình hoạt động tốt hơn trên toàn bộ dữ liệu. SVM tạo ra các lề mềm được gọi là *SVM lề mềm* (soft-margin SVM). Để phân biệt, SVM *thuần* trong chương trước được gọi là SVM lề cứng (hard-margin SVM).

Có hai cách xây dựng và giải quyết bài toán tối ưu SVM lề mềm. Cả hai đều mang lại những kết quả thú vị, có thể phát triển tiếp thành các thuật toán SVM phức tạp và hiệu quả hơn như sẽ thấy trong các chương sau. Cách thứ nhất là giải một bài toán tối ưu có ràng buộc thông qua việc giải bài toán đối ngẫu như với SVM lề cứng. Hướng giải quyết này là cơ sở cho phương pháp *SVM hạt nhân* áp dụng cho dữ liệu không thực sự tách biệt tuyến tính được đề cập trong chương tiếp theo. Cách giải quyết thứ hai là đưa về một bài toán tối ưu không ràng buộc, giải được bằng các phương pháp gradient descent. Nhờ đó, hướng giải quyết này có thể được áp dụng cho các bài toán quy mô lớn. Ngoài ra, trong cách giải này, chúng ta sẽ làm quen với một hàm mất mát mới có tên là *bản lề* (hinge). Hàm mất mát này có thể được mở rộng cho bài toán phân loại đa lớp được đề cập trong chương 29. Cách phát triển từ SVM lề mềm thành SVM đa lớp có thể được so sánh với cách phát triển từ hồi quy logistic thành hồi quy softmax.

27.2. Phân tích toán học

Như đã đề cập phía trên, để có một lề rộng hơn trong SVM lề mềm, ta cần hy sinh một vài điểm dữ liệu bằng cách chấp nhận cho chúng rơi vào vùng không an toàn. Tất nhiên, việc hy sinh này cần được hạn chế; nếu không, ta có thể tạo ra một biên cực lớn bằng cách hy sinh hầu hết các điểm. Vậy hàm mục tiêu nên là một sự kết hợp sao cho lề được tối đa và sự hy sinh được tối thiểu.



Hình 27.2. Giới thiệu các biến lỏng lẻo ξ_n. Với các điểm nằm trong khu vực an toàn, $\xi_n = 0$. Những điểm nằm trong vùng không an toàn nhưng vẫn đúng phía so với đường ranh giới (đường nét đứt đậm) tương ứng với các $0 < \xi_n < 1$, ví dụ $\mathbf{x}_2$. Những điểm nằm ngược phía lớp thực sự của chúng so với đường nét đứt đậm tương ứng $\xi_n > 1$, ví dụ như $\mathbf{x}_1$ và $\mathbf{x}_3$.

Giống SVM lề cứng, việc tối đa lề có thể đưa về việc tối thiểu $\|\mathbf{w}\|_2^2$. Để đong đếm sự hy sinh, chúng ta cùng quan sát Hình 27.2. Với mỗi điểm $\mathbf{x}_n$ trong tập huấn luyện, ta *giới thiệu* thêm một biến đo *sự hy sinh* ξ_n tương ứng. Biến này còn được gọi là *biến lỏng lẻo* (slack variable). Với những điểm $\mathbf{x}_n$ nằm trong vùng an toàn (nằm đúng vào màu nền tương ứng và nằm ngoài khu vực lề), $\xi_n = 0$, tức không có sự hy sinh nào xảy ra. Với mỗi điểm nằm trong vùng không an toàn như $\mathbf{x}_1, \mathbf{x}_2$ hay $\mathbf{x}_3$ ta cần có $\xi_i > 0$ để đo sự hy sinh. Đại lượng này cần tỉ lệ với khoảng cách từ vị trí vi phạm tương ứng tới biên giới an toàn (đường nét đứt mảnh tương ứng với lớp đó). Nhận thấy nếu $y_i = \pm 1$ là nhãn của $\mathbf{x}_i$ trong vùng không an toàn thì ξ_i có thể được định nghĩa bởi

$$\xi_i = |\mathbf{w}^T\mathbf{x}_i + b - y_i| \tag{27.1}$$

(Mẫu số $\|\mathbf{w}\|_2$ được lược bỏ vì ta chỉ cần một đại lượng tỉ lệ thuận.) Nhắc lại bài toán tối ưu cho SVM lề cứng:

$$\begin{aligned} (\mathbf{w}, b) &= \arg\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|_2^2 \\ \text{thoả mãn: } & y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1, \quad \forall n = 1, 2, \ldots, N \end{aligned} \tag{27.2}$$

Với SVM lề mềm, hàm mục tiêu sẽ có thêm một số hạng nữa giúp tối thiểu tổng sự hy sinh. Từ đó ta có hàm mục tiêu:

$$\frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{n=1}^{N} \xi_n \tag{27.3}$$

trong đó C là một hằng số dương. Hằng số C được dùng để điều chỉnh tầm quan trọng giữa độ rộng lề và sự hy sinh.

Điều kiện ràng buộc cũng được thay đổi so với SVM lề cứng. Với mỗi cặp dữ liệu $(\mathbf{x}_n, y_n)$, thay vì ràng buộc cứng $y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1$, ta sử dụng ràng buộc mềm:

$$y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1 - \xi_n \Leftrightarrow 1 - \xi_n - y_n(\mathbf{w}^T\mathbf{x}_n + b) \leq 0, \quad \forall n = 1, 2, \ldots, n$$

Và ràng buộc phụ $\xi_n \geq 0,\ \forall n = 1, 2, \ldots, N$. Tóm lại, ta có bài toán tối ưu chính cho SVM lề mềm như sau:

$$\begin{aligned}(\mathbf{w}, b, \xi) &= \arg\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{n=1}^{N}\xi_n \\ \text{thoả mãn: } 1 - \xi_n - y_n(\mathbf{w}^T\mathbf{x}_n + b) &\leq 0, \forall n = 1, 2, \ldots, N \\ -\xi_n &\leq 0,\ \forall n = 1, 2, \ldots, N\end{aligned} \tag{27.4}$$

Nhận xét:

- Nếu C nhỏ, việc sự hy sinh cao hay thấp không gây ảnh hưởng nhiều tới giá trị của hàm mục tiêu, thuật toán sẽ điều chỉnh sao cho $\|\mathbf{w}\|_2^2$ nhỏ nhất, tức lề lớn nhất, điều này dẫn tới $\sum_{n=1}^{N}\xi_n$ sẽ lớn theo vì vùng an toàn bị nhỏ đi. Ngược lại, nếu C quá lớn, để hàm mục tiêu đạt giá trị nhỏ nhất, thuật toán sẽ tập trung vào làm giảm $\sum_{n=1}^{N}\xi_n$. Trong trường hợp C rất rất lớn và hai lớp dữ liệu tách biệt tuyến tính, ta sẽ thu được $\sum_{n=1}^{N}\xi_n = 0$. Điều này đồng nghĩa với việc không có điểm nào phải hy sinh, nghiệm thu được cũng chính là nghiệm của SVM lề cứng. Nói cách khác, SVM lề cứng là một trường hợp đặc biệt của SVM lề mềm.

- Bài toán tối ưu (27.4) có thêm sự xuất hiện của các biến lỏng lẻo ξ_n. Các $\xi_n = 0$ ứng với những điểm dữ liệu nằm trong vùng an toàn. Các $0 < \xi_n \leq 1$ ứng với những điểm nằm trong vùng không an toàn nhưng vẫn được phân loại đúng, tức vẫn nằm về đúng phía so với đường phân chia. Các $\xi_n > 1$ tương ứng với các điểm bị phân loại sai.

- Hàm mục tiêu trong bài toán tối ưu (27.4) là một hàm lồi vì nó là tổng của hai hàm lồi: một hàm chuẩn và một hàm tuyến tính. Các hàm ràng buộc cũng là các hàm tuyến tính theo $(\mathbf{w}, b, \xi)$. Vì vậy bài toán tối ưu (27.4) là một bài toán lồi, hơn nữa còn có thể biểu diễn dưới dạng một bài toán quy hoạch toàn phương.

Tiếp theo, chúng ta sẽ giải bài toán tối ưu (27.4) bằng hai cách khác nhau.

27.3. Bài toán đối ngẫu Lagrange

Lưu ý rằng bài toán này có thể giải trực tiếp bằng các công cụ hỗ trợ quy hoạch toàn phương, nhưng giống như với SVM lề cứng, chúng ta sẽ quan tâm hơn tới bài toán đối ngẫu của nó.

Trước kết, ta cần kiểm tra tiêu chuẩn Slater của bài toán tối ưu lồi (27.4). Nếu tiêu chuẩn này thoả mãn, đối ngẫu mạnh sẽ thoả mãn, và ta có thể tìm nghiệm của bài toán tối ưu (27.4) thông qua hệ điều kiện KKT (xem Chương 25).

27.3.1. Kiểm tra tiêu chuẩn Slater

Rõ ràng là với mọi $n = 1, 2, \ldots, N$ và $(\mathbf{w}, b)$, ta luôn có thể tìm được các số dương $\xi_n, n = 1, 2, \ldots, N$, đủ lớn sao cho $y_n(\mathbf{w}^T\mathbf{x}_n + b) + \xi_n > 1,\ \forall n = 1, 2, \ldots, N$. Vì vậy, tồn tại điểm khả thi chặt cho bài toán và tiêu chuẩn Slater thỏa mãn.

27.3.2. Hàm Lagrange của bài toán SVM lề mềm

Hàm Lagrange cho bài toán (27.4) là

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{n=1}^{N}\xi_n + \sum_{n=1}^{N}\lambda_n(1 - \xi_n - y_n(\mathbf{w}^T\mathbf{x}_n + b)) - \sum_{n=1}^{N}\mu_n\xi_n \tag{27.5}$$

với $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \ldots, \lambda_N]^T \succeq 0$ và $\boldsymbol{\mu} = [\mu_1, \mu_2, \ldots, \mu_N]^T \succeq 0$ là các biến đối ngẫu Lagrange.

27.3.3. Bài toán đối ngẫu

Hàm số đối ngẫu của bài toán tối ưu (27.4) là:

$$g(\boldsymbol{\lambda}, \boldsymbol{\mu}) = \min_{\mathbf{w}, b, \boldsymbol{\xi}} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\lambda}, \boldsymbol{\mu})$$

Với mỗi cặp $(\boldsymbol{\lambda}, \boldsymbol{\mu})$, chúng ta đặc biệt quan tâm tới $(\mathbf{w}, b, \boldsymbol{\xi})$ thoả mãn điều kiện đạo hàm của hàm Lagrange bằng không:

$$\nabla_{\mathbf{w}}\mathcal{L} = 0 \Leftrightarrow \mathbf{w} = \sum_{n=1}^{N}\lambda_n y_n \mathbf{x}_n \tag{27.6}$$

$$\nabla_{b}\mathcal{L} = 0 \Leftrightarrow \sum_{n=1}^{N}\lambda_n y_n = 0 \tag{27.7}$$

$$\nabla_{\xi_n}\mathcal{L} = 0 \Leftrightarrow \lambda_n = C - \mu_n \tag{27.8}$$

Phương trình (27.8) chỉ ra rằng ta chỉ cần quan tâm tới những cặp $(\boldsymbol{\lambda}, \boldsymbol{\mu})$ sao cho $\lambda_n = C - \mu_n$. Từ đây cũng có thể suy ra $0 \leq \lambda_n, \mu_n \leq C, n = 1, 2, \ldots, N$. Thay các biểu thức này vào biểu thức hàm Lagrange (27.5), ta thu được hàm mục tiêu của bài toán đối ngẫu[67]:

$$g(\boldsymbol{\lambda}, \boldsymbol{\mu}) = \sum_{n=1}^{N}\lambda_n - \frac{1}{2}\sum_{n=1}^{N}\sum_{m=1}^{N}\lambda_n\lambda_m y_n y_m \mathbf{x}_n^T\mathbf{x}_m \tag{27.9}$$

Chú ý rằng hàm này không phụ thuộc vào $\boldsymbol{\mu}$ nhưng ta cần lưu ý ràng buộc (27.8), ràng buộc này và điều kiện không âm của $\boldsymbol{\lambda}$ có thể được viết gọn lại thành $0 \leq \lambda_n \leq C$, tức đã giảm được biến $\boldsymbol{\mu}$. Lúc này, bài toán đối ngẫu trở thành:

[67] Bạn đọc hãy coi đây như một bài tập nhỏ.

$$\boldsymbol{\lambda} = \arg\max_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda})$$

$$\text{thoả mãn:} \quad \sum_{n=1}^{N} \lambda_n y_n = 0 \tag{27.10}$$

$$0 \leq \lambda_n \leq C, \ \forall n = 1, 2, \ldots, N \tag{27.11}$$

Bài toán này giống bài toán đối ngẫu của SVM lề cứng, chỉ khác là có thêm ràng buộc λ_n bị chặn trên bởi C. Khi C rất lớn, ta có thể coi hai bài toán là như nhau. Ràng buộc (27.11) còn được gọi là *ràng buộc hộp* (box constraint) vì tập hợp các điểm $\boldsymbol{\lambda}$ thoả mãn ràng buộc này giống một hình hộp chữ nhật trong không gian nhiều chiều. Bài toán này cũng hoàn toàn giải được bằng các công cụ giải quy hoạch toàn phương thông thường, ví dụ CVXOPT. Sau khi tìm được $\boldsymbol{\lambda}$ của bài toán đối ngẫu, ta cần quay lại tìm nghiệm $(\mathbf{w}, b, \boldsymbol{\xi})$ của bài toán gốc. Trước hết, chúng ta cùng xem xét hệ điều kiện KKT và các tính chất của nghiệm.

27.3.4. Hệ điều kiện KKT

Hệ điều kiện KKT của bài toán tối ưu SVM lề mềm:

$$1 - \xi_n - y_n(\mathbf{w}^T\mathbf{x}_n + b) \leq 0 \tag{27.12}$$

$$-\xi_n \leq 0 \tag{27.13}$$

$$\lambda_n \geq 0 \tag{27.14}$$

$$\mu_n \geq 0 \tag{27.15}$$

$$\lambda_n(1 - \xi_n - y_n(\mathbf{w}^T\mathbf{x}_n + b)) = 0 \tag{27.16}$$

$$\mu_n \xi_n = 0 \tag{27.17}$$

$$\mathbf{w} = \sum_{n=1}^{N} \lambda_n y_n \mathbf{x}_n \tag{27.6}$$

$$\sum_{n=1}^{N} \lambda_n y_n = 0 \tag{27.7}$$

$$\lambda_n = C - \mu_n \tag{27.8}$$

với mọi $n = 1, 2, \ldots, N$.

Từ (27.6) và (27.8) ta thấy chỉ có những n ứng với $\lambda_n > 0$ mới đóng góp vào việc tính nghiệm $\mathbf{w}$ của bài toán SVM lề mềm. Tập hợp $\mathcal{S} = \{n : \lambda_n > 0\}$ được gọi là *tập hỗ trợ* (support set) và $\{\mathbf{x}_n, n \in \mathcal{S}\}$ được gọi là tập các vector hỗ trợ.

Khi $\lambda_n > 0$, (27.16) chỉ ra rằng:

$$y_n(\mathbf{w}^T\mathbf{x}_n + b) = 1 - \xi_n \tag{27.18}$$

Nếu $0 < \lambda_n < C$, (27.8) nói rằng $\mu_n = C - \lambda_n > 0$. Kết hợp với (27.17), ta thu được $\xi_n = 0$. Tiếp tục kết hợp với (27.18), ta suy ra $y_n(\mathbf{w}^T\mathbf{x}_n + b) = 1$, hay nói cách khác $\mathbf{w}^T\mathbf{x}_n + b = y_n, \ \forall n : 0 < \lambda_n < C$.

Tóm lại, khi $0 < \lambda_n < C$, các điểm $\mathbf{x}_n$ nằm chính xác trên hai đường thẳng hỗ trợ (hai đường nét đứt mảnh trong Hình 27.2). Tương tự như SVM lề cứng, giá trị b có thể được tính theo công thức:

$$b = \frac{1}{N_{\mathcal{M}}} \sum_{m \in \mathcal{M}} \left(y_m - \mathbf{w}^T \mathbf{x}_m\right) \tag{27.19}$$

với $\mathcal{M} = \{m : 0 < \lambda_m < C\}$ và $N_{\mathcal{M}}$ là số phần tử của $\mathcal{S}$. Nghiệm của bài toán SVM lề mềm được cho bởi (27.6) và (27.19).

Nghiệm của bài toán SVM lề mềm

$$\mathbf{w} = \sum_{m \in \mathcal{S}} \lambda_m y_m \mathbf{x}_m \tag{27.20}$$

$$b = \frac{1}{N_{\mathcal{M}}} \sum_{n \in \mathcal{M}} (y_n - \mathbf{w}^T \mathbf{x}_n) = \frac{1}{N_{\mathcal{M}}} \sum_{n \in \mathcal{M}} \left(y_n - \sum_{m \in \mathcal{S}} \lambda_m y_m \mathbf{x}_m^T \mathbf{x}_n \right) \tag{27.21}$$

Với $\lambda_n = C$, từ (27.8) và (27.16) ta suy ra $y_n(\mathbf{w}^T \mathbf{x}_n + b) = 1 - \xi_n \leq 1$. Điều này nghĩa là những điểm ứng với $\lambda_n = C$ nằm giữa hai đường hỗ trợ hoặc nằm trên chúng. Như vậy, dựa trên các giá trị của λ_n ta có thể xác định được vị trí tương đối của $\mathbf{x}_n$ so với hai đường hỗ trợ.

Mục đích cuối cùng là xác định nhãn cho một điểm mới $\mathbf{x}$. Vì vậy, ta quan tâm hơn tới cách xác định giá trị của biểu thức sau đây:

$$\mathbf{w}^T \mathbf{x} + b = \sum_{m \in \mathcal{S}} \lambda_m y_m \mathbf{x}_m^T \mathbf{x} + \frac{1}{N_{\mathcal{M}}} \sum_{n \in \mathcal{M}} \left(y_n - \sum_{m \in \mathcal{S}} \lambda_m y_m \mathbf{x}_m^T \mathbf{x}_n \right) \tag{27.22}$$

Biểu thức này có thể được xác định trực tiếp thông qua các điểm dữ liệu huấn luyện. Ta không cần thực hiện việc tính $\mathbf{w}$ và b. Nếu có thể tính các tích vô hướng $\mathbf{x}_m^T \mathbf{x}$ và $\mathbf{x}_m^T \mathbf{x}_n$, ta sẽ xác định được bộ phân loại. Quan sát này rất quan trọng và là ý tưởng chính cho *SVM hạt nhân* được trình bày trong chương tiếp theo.

27.4. Bài toán tối ưu không ràng buộc cho SVM lề mềm

Trong mục này, chúng ta sẽ biến đổi bài toán tối ưu có ràng buộc (27.4) về bài toán tối ưu không ràng buộc có thể giải được bằng các phương pháp gradient descent. Đây cũng là ý tưởng chính cho *SVM đa lớp* được trình bày trong Chương 29.

27.4.1. Bài toán tối ưu không ràng buộc tương đương

Để ý rằng điều kiện ràng buộc thứ nhất:

$$1 - \xi_n - y_n(\mathbf{w}^T\mathbf{x} + b) \leq 0 \Leftrightarrow \xi_n \geq 1 - y_n(\mathbf{w}^T\mathbf{x} + b) \tag{27.23}$$

Kết hợp với điều kiện $\xi_n \geq 0$ ta thu được bài toán ràng buộc tương đương với bài toán (27.4) như sau:

$$\begin{gathered}(\mathbf{w}, b, \xi) = \arg\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{n=1}^{N}\xi_n \\ \text{thoả mãn: } \xi_n \geq \max(0, 1 - y_n(\mathbf{w}^T\mathbf{x} + b)),\ \forall n = 1, 2, \ldots, N\end{gathered} \tag{27.24}$$

Để đưa bài toán (27.24) về dạng không ràng buộc, chúng ta sẽ chứng minh nhận xét sau đây bằng phương pháp phản chứng: Nếu $(\mathbf{w}, b, \xi)$ là điểm tối ưu của bài toán (27.24) thì

$$\xi_n = \max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)),\ \forall n = 1, 2, \ldots, N \tag{27.25}$$

Thật vậy, giả sử ngược lại, tồn tại n sao cho:

$$\xi_n > \max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)),$$

chọn $\xi'_n = \max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b))$, ta sẽ thu được một giá trị thấp hơn của hàm mục tiêu, trong khi tất cả các ràng buộc vẫn được thoả mãn. Điều này mâu thuẫn với việc hàm mục tiêu đã đạt giá trị nhỏ nhất tương ứng với ξ_n! Điều mâu thuẫn này chỉ ra rằng nhận xét (27.25) là chính xác.

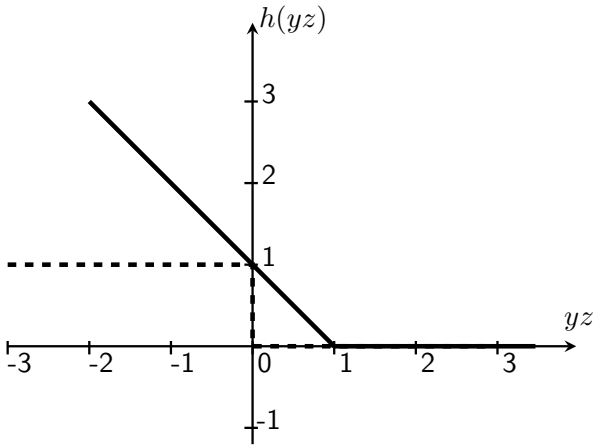
Khi đó, bằng cách thay toàn bộ các giá trị của ξ_n trong (27.25) vào hàm mục tiêu, ta thu được bài toán tối ưu

$$\begin{gathered}(\mathbf{w}, b, \xi) = \arg\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{n=1}^{N}\max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)) \\ \text{thoả mãn: } \xi_n = \max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)),\ \forall n = 1, 2, \ldots, N\end{gathered} \tag{27.26}$$

Từ đây ta thấy biến số ξ không xuất hiện trong hàm mục tiêu, vì vậy điều kiện ràng buộc có thể được bỏ qua:

$$(\mathbf{w}, b) = \arg\min_{\mathbf{w},b}\left\{\frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{n=1}^{N}\max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)) \triangleq J(\mathbf{w}, b)\right\} \tag{27.27}$$

Đây là một bài toán tối ưu không ràng buộc với hàm mất mát $J(\mathbf{w}, b)$. Bài toán này có thể được giải bằng các phương pháp gradient descent. Nhưng trước hết cùng xem xét hàm số này từ một góc nhìn khác bằng cách sử dụng *hàm mất mát bản lề* (hinge loss).



Hình 27.3. Mất mát bản lề (nét liền) và mất mát không-một (nét đứt). Với mất mát không-một, những điểm nằm xa đường hỗ trợ (hoành độ bằng 1) và đường phân chia (hoành độ bằng 0) đều mang lại mất mát bằng một. Trong khi đó, với mất mát bản lề, những điểm ở xa về phía trái gây ra mất mát nhiều hơn.

27.4.2. Mất mát bản lề

Nhắc lại hàm entropy chéo: Với mỗi cặp hệ số $(\mathbf{w}, b)$ và dữ liệu $(\mathbf{x}_n, y_n)$, đặt $a_n = \sigma(\mathbf{w}^T\mathbf{x}_n + b)$ (hàm sigmoid). Hàm entropy chéo được định nghĩa là:

$$J_n^1(\mathbf{w}, b) = -(y_n \log(a_n) + (1 - y_n)\log(1 - a_n)) \tag{27.28}$$

Hàm số này đạt giá trị nhỏ nếu xác suất a_n gần với y_n $(0 < a_n < 1, y_n \in \{0, 1\})$.

Ở đây, chúng ta làm quen với một hàm số khác cũng được sử dụng nhiều trong các hệ thống phân loại. Hàm số này có dạng

$$J_n(\mathbf{w}, b) = \max(0, 1 - y_n z_n)$$

Hàm này có tên là *mất mát bản lề* (hinge loss). Trong đó, $z_n = \mathbf{w}^T\mathbf{x}_n + b$ còn được gọi là *điểm số* (score) của $\mathbf{x}_n$ ứng với cặp hệ số $(\mathbf{w}, b)$, $y_n = \pm 1$ là nhãn của $\mathbf{x}_n$. Hình 27.3 mô tả đồ thị hàm mất mát bản lề[68] $f(yz) = \max(0, 1 - yz)$ và so sánh với hàm *mất mát không-một* (zero-one loss). Hàm mất mát không-một trả về không nếu một điểm được phân loại đúng và trả về một nếu điểm đó bị phân loại sai. Như vậy, mất mát không-một là hàm đếm số điểm bị phân loại sai của tập huấn luyện. Trong Hình 27.5, biến số là yz là tích của đầu ra mong muốn y và điểm số z. Những điểm ở phía phải của trục tung ứng với những điểm được phân loại đúng, tức z tìm được cùng dấu với y. Những điểm ở phía trái của trục tung ứng với các điểm bị phân loại sai. Ta có các nhận xét sau đây:

- Với mất mát không-một, các điểm dữ liệu có điểm số ngược dấu với đầu ra mong muốn $(yz < 0)$ sẽ gây ra mất mát như nhau và đều bằng một, bất kể chúng ở gần hay xa đường ranh giới (trục tung). Đây là một hàm rời rạc, rất khó tối ưu và không giúp đo đếm sự hy sinh nếu một điểm nằm quá xa so với đường hỗ trợ.

[68] Đồ thị của hàm số này có dạng chiếc bản lề.

- Với mất mát bản lề, những điểm nằm trong vùng an toàn ứng với $yz \geq 1$ sẽ không gây ra mất mát gì. Những điểm nằm giữa đường hỗ trợ của lớp tương ứng và đường ranh giới ứng với $0 < y < 1$ sẽ gây ra một mất mát nhỏ (nhỏ hơn một). Những điểm bị phân loại lỗi, tức $yz < 0$ sẽ gây ra mất mát lớn hơn. Vì vậy, khi tối thiểu hàm mất mát, ta sẽ hạn chế được những điểm bị phân loại lỗi và sang lớp kia quá nhiều. Đây chính là một ưu điểm của mất mát bản lề.

- Mất mát bản lề là một hàm liên tục, và *có đạo hàm tại gần như mọi nơi* (almost everywhere differentiable) trừ điểm có hoành độ bằng 1. Ngoài ra, đạo hàm của hàm này theo yz cũng rất dễ xác định: bằng -1 tại các điểm nhỏ hơn 1 và bằng 0 tại các điểm lớn hơn 1. Tại 1, ta có thể coi đạo hàm của nó bằng 0.

27.4.3. Xây dựng hàm mất mát

Xét bài toán SVM lề mềm sử dụng mất mát bản lề, với mỗi cặp $(\mathbf{w}, b)$, đặt

$$L_n(\mathbf{w}, b) = \max(0, 1 - y_n z_n) = \max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)) \tag{27.29}$$

Lấy trung bình cộng của các mất mát này trên toàn tập huấn luyện ta được

$$L(\mathbf{w}, b) = \frac{1}{N}\sum_{n=1}^{N} L_n = \frac{1}{N}\sum_{n=1}^{N} \max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b))$$

Trong trường hợp dữ liệu hai lớp tách biệt tuyến tính, giá trị tối ưu tìm được của $L(\mathbf{w}, b)$ sẽ bằng 0. Điều này nghĩa là:

$$1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) \leq 0,\ \forall n = 1, 2, \ldots, N \tag{27.30}$$

Nhân cả hai vế với một hằng số $a > 1$ ta có:

$$a - y_n(a\mathbf{w}^T\mathbf{x}_n + ab) \leq 0,\ \forall n = 1, 2, \ldots, N \tag{27.31}$$

$$\Rightarrow 1 - y_n(a\mathbf{w}^T\mathbf{x}_n + ab) \leq 1 - a < 0,\ \forall n = 1, 2, \ldots, N \tag{27.32}$$

Điều này chỉ ra $(a\mathbf{w}, ab)$ cũng là nghiệm của bài toán. Nếu không có thêm ràng buộc, bài toán có thể dẫn tới nghiệm không ổn định vì $\mathbf{w}$ và b có thể lớn tuỳ ý!

Để tránh hiện tượng này, chúng ta cần thêm một số hạng kiểm soát vào $L(\mathbf{w}, b)$ giống như cách làm để tránh quá khớp trong mạng neuron. Lúc này, ta sẽ có hàm mất mát tổng cộng:

$$J(\mathbf{w}, b) = L(\mathbf{w}, b) + \lambda R(\mathbf{w}, b)$$

với λ là một số dương, gọi là tham số kiểm soát, hàm $R()$ giúp hạn chế việc các hệ số $(\mathbf{w}, b)$ quá lớn. Có nhiều cách chọn hàm $R()$, nhưng cách phổ biến nhất là dùng chuẩn ℓ_2, khi đó hàm mất mát của SVM lề mềm trở thành:

$$J(\mathbf{w}, b) = \frac{1}{N}\left(\underbrace{\sum_{n=1}^{N} \max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b))}_{\text{mất mát bản lề}} + \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|_2^2}_{\text{kiểm soát}}\right) \tag{27.33}$$

Kỹ thuật này tương đương với kỹ thuật suy giảm trọng số trong mạng neuron. Suy giảm trọng số không được áp dụng lên hệ số điều chỉnh b.

Ta thấy rằng hàm mất mát (27.33) tương đương hàm mất mát (27.27) với $\lambda = \frac{1}{C}$.

Trong phần tiếp theo của mục này, chúng ta sẽ quan tâm tới bài toán tối ưu hàm mất mát được cho trong (27.33). Trước hết, đây là một hàm lồi theo $\mathbf{w}, b$ vì các lý do sau:

- $1 - y_n(\mathbf{w}^T\mathbf{x}_n + b)$ là một hàm lồi vì nó tuyến tính theo $\mathbf{w}, b$. Hàm lấy giá trị lớn hơn trong hai hàm lồi là một hàm lồi. Vì vậy, mất mát bản lề là một hàm lồi.
- Chuẩn là một hàm lồi.
- Tổng của hai hàm lồi là một hàm lồi.

Vì hàm mất mát là lồi, các thuật toán gradient descent với tốc độ học phù hợp sẽ giúp tìm nghiệm của bài toán một cách hiệu quả.

27.4.4. Tối ưu hàm mất mát

Để sử dụng gradient descent, chúng ta cần tính đạo hàm của hàm mất mát theo $\mathbf{w}$ và b.

Đạo hàm của mất mát bản lề không quá phức tạp:

$$\nabla_{\mathbf{w}}\left(\max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b))\right) = \begin{cases} -y_n\mathbf{x}_n & \text{nếu} \quad 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 0 \\ \mathbf{0} & \text{o.w.} \end{cases}$$

$$\nabla_{b}\left(\max(0, 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b))\right) = \begin{cases} -y_n & \text{nếu} \quad 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 0 \\ 0 & \text{o.w.} \end{cases}$$

Phần kiểm soát cũng có đạo hàm tương đối đơn giản:

$$\nabla_{\mathbf{w}}\left(\frac{\lambda}{2}\|\mathbf{w}\|_2^2\right) = \lambda\mathbf{w}; \qquad \nabla_{b}\left(\frac{\lambda}{2}\|\mathbf{w}\|_2^2\right) = 0$$

Khi sử dụng stochastic gradient descent trên từng điểm dữ liệu, nếu $1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) < 0$, ta không cần cập nhật và chuyển sang điểm tiếp theo. Ngược lại biểu thức cập nhật cho $\mathbf{w}, b$ được cho bởi:

$$\begin{array}{llll} \mathbf{w} \leftarrow \mathbf{w} - \eta(-y_n\mathbf{x}_n + \lambda\mathbf{w}); & b \leftarrow b + \eta y_n & \text{nếu} & 1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 0) \\ \mathbf{w} \leftarrow \mathbf{w} - \eta\lambda\mathbf{w}; & b \leftarrow b & \text{o.w.} & \end{array}$$

với η là tốc độ học. Với mini-batch gradient descent hoặc batch gradient descent, các biểu thức đạo hàm trên đây hoàn toàn có thể được lập trình bằng các kỹ thuật vector hóa như chúng ta sẽ thấy trong mục tiếp theo.

27.5. Lập trình với SVM lề mềm

Trong mục này, nghiệm của một bài toán SVM lề mềm được tìm bằng ba cách khác nhau: sử dụng thư viện sklearn, giải bài toán đối ngẫu bằng CVXOPT, và giải bài toán tối ưu không ràng buộc bằng gradient descent. Giá trị C được sử dụng là 100. Nếu mọi tính toán từ đầu chương là chính xác, nghiệm của ba cách làm này sẽ gần giống nhau, sự khác nhau có thể đến từ sai số tính toán. Chúng ta cũng sẽ thay C bởi những giá trị khác nhau và quan sát sự thay đổi của lề.

Khai báo thư viện và tạo dữ liệu giả:

```
from __future__ import print_function
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(22)

means = [[2, 2], [4, 2]]
cov = [[.7, 0], [0, .7]]
N = 20 # number of samplers per class
X0 = np.random.multivariate_normal(means[0], cov, N) # each row is a data
    point
X1 = np.random.multivariate_normal(means[1], cov, N)
X = np.concatenate((X0, X1))
y = np.concatenate((np.ones(N), -np.ones(N)))
```

Hình 27.4 minh hoạ các điểm dữ liệu của hai lớp. Hai lớp dữ liệu gần tách biệt tuyến tính.

27.5.1. Giải bài toán bằng thư viện sklearn

```
from sklearn.svm import SVC
C = 100
clf = SVC(kernel = 'linear', C = C)
clf.fit(X, y)
w_sklearn = clf.coef_.reshape(-1, 1)
b_sklearn = clf.intercept_[0]
print(w_sklearn.T, b_sklearn)
```

Kết quả:

```
w_sklearn =  [[-1.87461946 -1.80697358]]
b_sklearn =  8.49691190196
```

27.5.2. Tìm nghiệm bằng cách giải bài toán đối ngẫu

Đoạn mã dưới đây tương tự với việc giải bài toán SVM lề cứng có thêm chặn trên của các nhân tử Lagrange:

```
from cvxopt import matrix, solvers
# build K
V = np.concatenate((X0, -X1), axis = 0) # V[n,:] = y[n]*X[n]
K = matrix(V.dot(V.T))
p = matrix(-np.ones((2*N, 1)))
# build A, b, G, h
G = matrix(np.vstack((-np.eye(2*N), np.eye(2*N))))
h = np.vstack((np.zeros((2*N, 1)), C*np.ones((2*N, 1))))
h = matrix(np.vstack((np.zeros((2*N, 1)), C*np.ones((2*N, 1)))))
A = matrix(y.reshape((-1, 2*N)))
b = matrix(np.zeros((1, 1)))
solvers.options['show_progress'] = False
sol = solvers.qp(K, p, G, h, A, b)

l = np.array(sol['x']).reshape(2*N) # lambda vector

# support set
S = np.where(l > 1e-5)[0]
S2 = np.where(l < .999*C)[0]
# margin set
M = [val for val in S if val in S2] # intersection of two lists

VS = V[S]           # shape (NS, d)
lS = l[S]           # shape (NS,)
w_dual = lS.dot(VS) # shape (d,)
yM = y[M]           # shape(NM,)
XM = X[M]           # shape(NM, d)
b_dual = np.mean(yM - XM.dot(w_dual)) # shape (1,)
print('w_dual = ', w_dual)
print('b_dual = ', b_dual)
```

Kết quả:

```
w_dual =  [-1.87457279 -1.80695039]
b_dual =  8.49672109814
```

Kết quả này gần giống với kết quả tìm được bằng sklearn.

27.5.3. Tìm nghiệm bằng giải bài toán tối ưu không ràng buộc

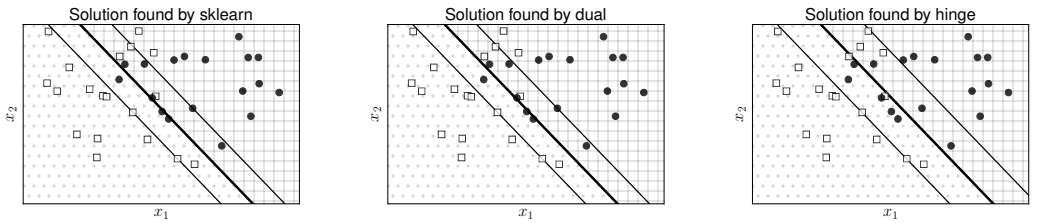
Trong phương pháp này, chúng ta cần tính gradient của hàm mất mát. Như thường lệ, cần kiểm chứng tính chính xác của đạo hàm này. Chú ý rằng trong phương pháp này, ta cần dùng tham số `lam = 1/C`. Trước hết viết các hàm tính giá trị hàm mất mát và đạo hàm theo $\mathbf{w}$ và b:

```
lam = 1./C
def loss(X, y, w, b):
    """
    X.shape = (2N, d), y.shape = (2N,), w.shape = (d,), b is a scalar
    """
    z = X.dot(w) + b # shape (2N,)
    yz = y*z
    return (np.sum(np.maximum(0, 1 - yz)) + .5*lam*w.dot(w))/X.shape[0]

def grad(X, y, w, b):
    z = X.dot(w) + b # shape (2N,)
    yz = y*z         # element wise product, shape (2N,)
    active_set = np.where(yz <= 1)[0] # consider 1 - yz >= 0 only
    _yX = - X*y[:, np.newaxis]   # each row is y_n*x_n
    grad_w = (np.sum(_yX[active_set], axis = 0) + lam*w)/X.shape[0]
    grad_b = (-np.sum(y[active_set]))/X.shape[0]
    return (grad_w, grad_b)

def num_grad(X, y, w, b):
    eps = 1e-10
    gw = np.zeros_like(w)
    gb = 0
    for i in xrange(len(w)):
        wp = w.copy()
        wm = w.copy()
        wp[i] += eps
        wm[i] -= eps
        gw[i] = (loss(X, y, wp, b) - loss(X, y, wm, b))/(2*eps)
    gb = (loss(X, y, w, b + eps) - loss(X, y, w, b - eps))/(2*eps)
    return (gw, gb)

w = .1*np.random.randn(X.shape[1])
b = np.random.randn()
(gw0, gb0) = grad(X, y, w, b)
(gw1, gb1) = num_grad(X, y, w, b)
print('grad_w difference = ', np.linalg.norm(gw0 - gw1))
print('grad_b difference = ', np.linalg.norm(gb0 - gb1))
```

Kết quả:

```
grad_w difference =  1.27702840067e-06
grad_b difference =  4.13701854995e-08
```

Sự sai khác giữa hai cách tính gradient khá nhỏ; ta có thể tin tưởng sử dụng hàm `grad` khi thực hiện gradient descent.



(a) **(b)** **(c)**

Hình 27.4. Các đường phân chia tìm được bởi ba cách khác nhau: a) Thư viện sklearn, b) Giải bài toán đối ngẫu bằng CVXOPT, c) Hàm mất mát bản lề. Các kết quả tìm được gần giống nhau.

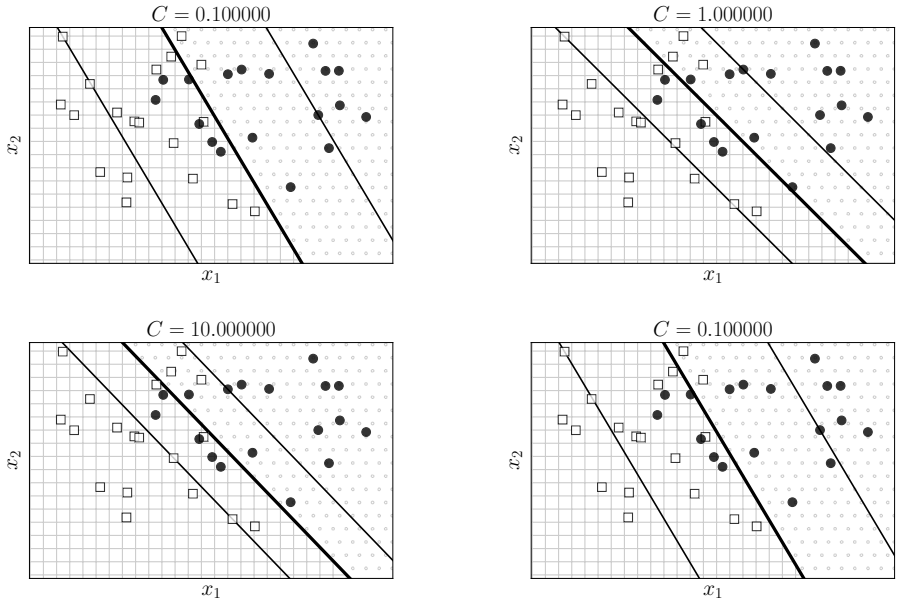
Đoạn mã dưới đây trình bày cách cập nhật nghiệm bằng gradient descent:

```
def softmarginSVM_gd(X, y, w0, b0, eta):
    w, b, it = w0, b0, 0
    while it < 10000:
        it = it + 1
        (gw, gb) = grad(X, y, w, b)
        w -= eta*gw
        b -= eta*gb
        if (it % 1000) == 0:
            print('iter %d' %it + ' loss: %f' %loss(X, y, w, b))
    return (w, b)

w0 = .1*np.random.randn(X.shape[1])
b0 = .1*np.random.randn()
lr = 0.05
(w_hinge, b_hinge) = softmarginSVM_gd(X, y, w0, b0, lr)
print('w_hinge = ', w_dual)
print('b_hinge = ', b_dual)
```

Kết quả:

```
iter 1000 loss: 0.436460
iter 2000 loss: 0.405307
iter 3000 loss: 0.399860
iter 4000 loss: 0.395440
iter 5000 loss: 0.394562
iter 6000 loss: 0.393958
iter 7000 loss: 0.393805
iter 8000 loss: 0.393942
iter 9000 loss: 0.394005
iter 10000 loss: 0.393758
w_hinge =  [-1.87457279 -1.80695039]
b_hinge =  8.49672109814
```

Ta thấy rằng `loss` giảm dần và hội tụ theo thời gian. Nghiệm này cũng gần giống nghiệm tìm được bằng sklearn và CVXOPT. Hình 27.4 mình hoạ các nghiệm tìm



Hình 27.5. Ảnh hưởng của C lên nghiệm của SVM lề mềm. C càng lớn thì biên càng nhỏ và ngược lại.

được bằng cả ba phương pháp. Ta thấy rằng các nghiệm tìm được gần như giống nhau.

27.5.4. Ảnh hưởng của C lên nghiệm

Hình 27.5 minh hoạ nghiệm tìm được bằng sklearn với các giá trị C khác nhau. Quan sát thấy khi C càng lớn, biên càng nhỏ đi. Điều này phù hợp với các suy luận ở đầu chương.

27.6. Tóm tắt và thảo luận

- SVM thuần (SVM lề cứng) hoạt động không hiệu quả khi có nhiễu ở gần ranh giới hoặc khi dữ liệu giữa hai lớp gần tách biệt tuyến tính. SVM lề mềm có thể giúp khắc phục điểm này.

- Trong SVM lề mềm, chúng ta chấp nhận lỗi xảy ra ở một vài điểm dữ liệu. Lỗi này được xác định bằng khoảng cách từ điểm đó tới đường hỗ trợ tương ứng. Bài toán tối ưu sẽ tối thiểu lỗi này bằng cách sử dụng thêm các biến lỏng lẻo. Có hai cách khác nhau giải bài toán tối ưu.

- Cách thứ nhất là giải bài toán đối ngẫu. Bài toán đối ngẫu của SVM lề mềm rất giống với bài toán đối ngẫu của SVM lề cứng ngoại trừ việc có thêm ràng buộc chặn trên của các nhân tử Laggrange. Ràng buộc này còn được gọi là ràng buộc hộp.

- Cách thứ hai là đưa bài toán về dạng không ràng buộc dựa trên mất mát bản lề. Trong phương pháp này, hàm mất mát thu được là một hàm lồi và có thể giải hiệu quả bằng các phương pháp gradient descent.

- SVM lề mềm yêu cầu chọn hằng số C. Hướng tiếp cận này còn được gọi là C-SVM. Ngoài ra, còn có một hướng tiếp cận khác cũng hay được sử dụng, gọi là ν-SVM [SSWB00].

- Mã nguồn trong chương này có thể được tìm thấy tại https://goo.gl/PuWxba.

- LIBSVM là một thư viện SVM phổ biến (https://goo.gl/Dt7o7r).

- Đọc thêm: L. Rosasco *et al.*,. *Are Loss Functions All the Same?* (https://goo.gl/QH2Cgr). *Neural Computation.*2004 [RDVC+04].